

Agent 记忆系统设计：短期、长期到知识图谱

原创 Jameszyh James的成长日记 2026年4月10日 14:19 广东

大家好，我是James。

上一篇我们拆解了 ReAct 模式——Agent 如何边想边做、随时调整。但你有没有注意到一个问题：ReAct 的每一轮循环，Agent 都知道「之前发生了什么」，可一旦会话结束，它就什么都不记得了。

你跟 AI 聊了两小时，把背景说了一遍，下次打开又要重新说一遍。你让它记住你的偏好，它答应了，但第二天它又变成了"陌生人"。

这不是模型记忆力差，是架构里根本没有「记忆系统」。今天这篇，我们就来彻底搞清楚 Agent 的记忆怎么设计。

一句话定义

Agent 记忆系统 = 让 AI 像人一样，把「刚刚发生的」「学过的」「长期积累的」分层管理。

类比：人类的记忆分三层——工作记忆（当前对话的上下文，几分钟内）、情节记忆（某件具体的事，比如「上周我们讨论过 XX 方案」）、语义记忆（知识图谱，比如「TypeScript 的类型系统是这样的」）。

AI 的记忆系统设计和这个完全对应：

人类记忆	Agent 对应	实现方式
工作记忆	短期记忆	Context Window（消息列表）
情节记忆	长期记忆	向量数据库（语义检索）
语义记忆	知识图谱	结构化知识存储（图数据库）

第一层：短期记忆——Context Window 的正确用法

短期记忆就是 Context Window 里的消息列表，但原样塞满不是最优解。

大多数开发者的实现是直接追加：



```
// ❌ 原始实现：无脑追加，必然 OOM
const messages: Message[] = [];

async function chat(userInput: string) {
  messages.push({ role: "user", content: userInput });

  const response = await llm.invoke(messages); // 越来越长，最终爆 context

  messages.push({ role: "assistant", content: response.content });
  return response.content;
}
```

问题很明显：聊 20 轮之后，Context Window 满了，要么报错，要么模型开始"遗忘"早期内容。

正确做法：滑动窗口 + 摘要压缩，保留近期上下文的同时不丢失历史。

```
import { ChatOpenAI } from "@langchain/openai";
import { SystemMessage, HumanMessage, AIMessage } from "@langchain/core/messages";

const llm = new ChatOpenAI({ model: "gpt-4o-mini" });

// 核心：动态管理消息列表
async function manageHistory(
  history: Message[],
  maxTokens = 4000
): Promise<Message[]> {
  const currentTokens = estimateTokens(history);

  if (currentTokens <= maxTokens) {
    return history; // 还没满，原样返回
  }

  // 保留最近 10 条（5轮对话），防止丢失即时上下文
  const recent = history.slice(-10);
  const older = history.slice(0, -10);

  // 用模型压缩历史，比规则截断效果好 30%
  const summary = await llm.invoke([
    new SystemMessage(
      "将以下对话历史压缩成 200 字以内的摘要，保留关键决策、用户偏好和重要结论"
    ),
    new HumanMessage(
      older.map((m) => `${m.role}: ${m.content}`).join("\n")
    ),
  ]);

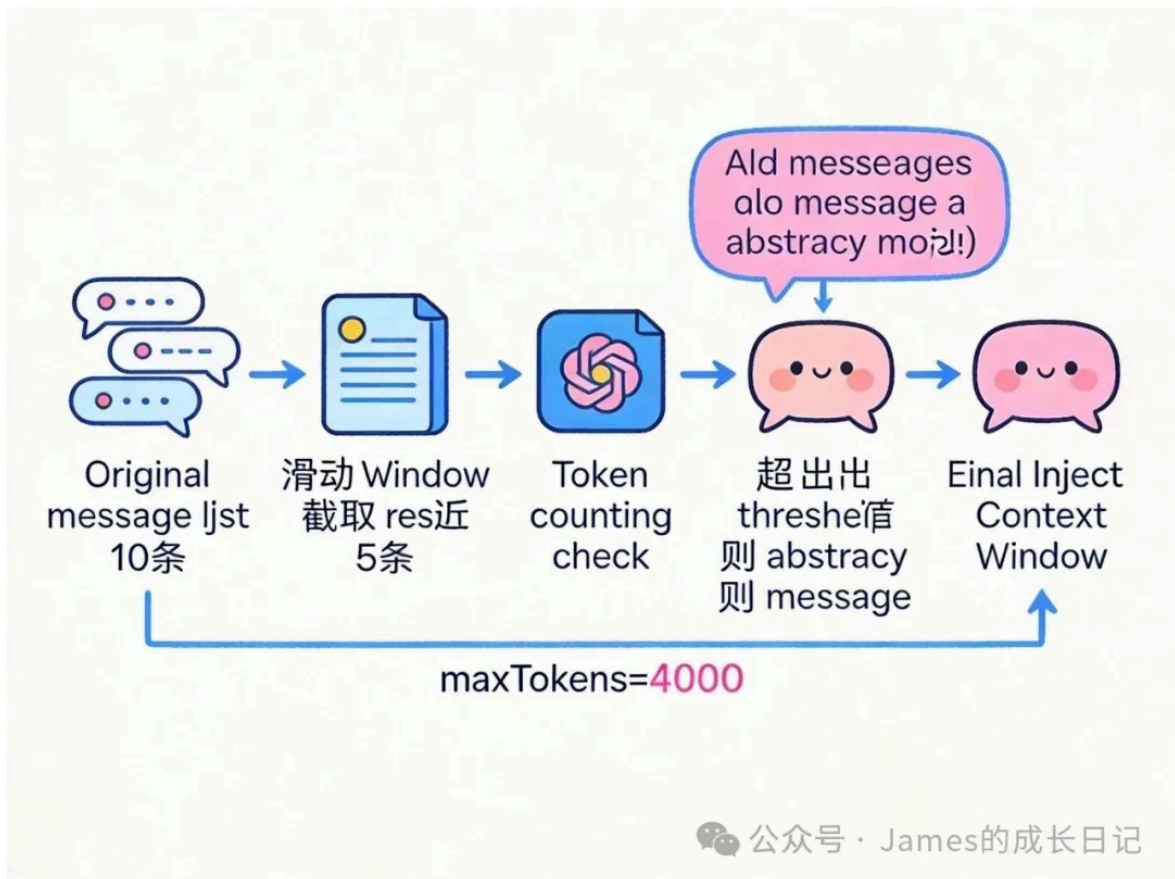
  // 摘要作为系统消息放最前面
```

```

return [
  newSystemMessage(`对话历史摘要: ${summary.content}`),
  ...recent,
];
}

// 粗估 token 数 (4个字符约等于1个token)
function estimateTokens(messages: Message[]): number {
  return messages.reduce((sum, m) => sum + m.content.length / 4, 0);
}

```



核心：短期记忆不是越长越好，滑动窗口 + 摘要压缩是平衡成本和质量的正确姿势。

第二层：长期记忆——向量数据库实现语义检索

长期记忆解决的问题是：「我一个月前告诉过你的事，你现在能不能想起来？」

实现原理很直接——把历史对话或重要信息向量化存储，需要时按语义相似度检索。

关键点在于：存什么、什么时候存、存多少。

```

import { OpenAIEmbeddings } from "@langchain/openai";
import { MemoryVectorStore } from "langchain/vectorstores/memory";
import { Document } from "@langchain/core/documents";

```

```

// 初始化向量存储（生产环境用 Pinecone / Weaviate / Chroma）
const embeddings = newOpenAIEmbeddings();
const vectorStore = newMemoryVectorStore(embeddings);

// ✅ 选择性存储：只存有价值的信息
asyncfunctionsaveToLongTermMemory(
  content: string,
  metadata: {
    type: "preference" | "fact" | "decision" | "task";
    importance: "high" | "medium" | "low";
    userId: string;
  }
) {
  // 低重要度的内容不存，节省存储和检索噪音
  if (metadata.importance === "low") return;

  await vectorStore.addDocuments([
    newDocument({
      pageContent: content,
      metadata: {
        ...metadata,
        timestamp: Date.now(),
        // 重要！存储时间戳，旧的记忆权重重要打折
      },
    }),
  ]);
}

// ✅ 检索时加时间衰减：越近的记忆越相关
asyncfunctionrecallMemory(query: string, userId: string) {
  const results = await vectorStore.similaritySearchWithScore(
    query,
    5, // 取最相似的 5 条
    { userId } // 只检索当前用户的记忆
  );

  // 时间衰减：30天前的记忆相关性打 0.7 折
  const now = Date.now();
  const decayedResults = results.map(([doc, score]) => {
    const ageInDays = (now - doc.metadata.timestamp) / (1000 * 60 * 60 * 24);
    const decayFactor = ageInDays > 30 ? 0.7 : 1.0;
    return { doc, score: score * decayFactor };
  });

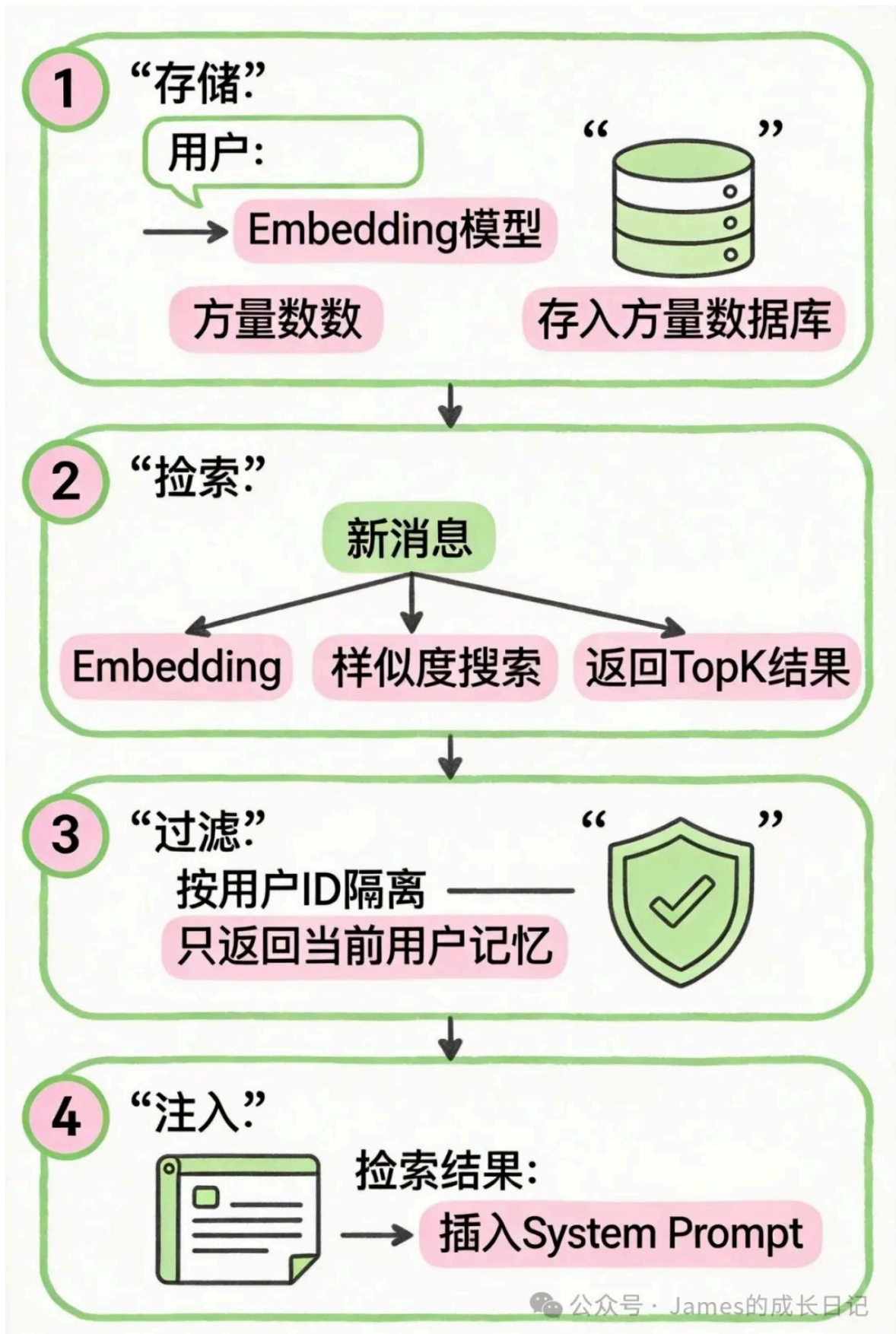
  // 只返回相关性 > 0.7 的结果，避免噪音
  return decayedResults
    .filter(({ score }) => score > 0.7)
    .map(({ doc }) => doc.pageContent);
}

```

```
// 在 Agent 回答前，先检索相关记忆注入 context
async function agentWithMemory(userInput: string, userId: string) {
  const memories = await recallMemory(userInput, userId);

  const systemPrompt = memories.length > 0
    ? `你记得关于这个用户的以下信息：\n${memories.join("\n")}\n\n基于这些记忆回答问题。`
    : "你是一个助手。";

  return llm.invoke([
    new SystemMessage(systemPrompt),
    new HumanMessage(userInput),
  ]);
}
```



核心：长期记忆不是把所有对话都存进去，选择性存储 + 时间衰减才能保持信噪比。

第三层：知识图谱——结构化知识的极致形态

知识图谱解决的是「关系」问题——不只是记住事实，还要记住事实之间的关联。

举个例子：「用户喜欢 React」「用户在做 AI 项目」「React 有 AI SDK」——如果这三条是孤立存储的，Agent 无法推导出「可以向用户推荐 Vercel AI SDK」。但如果存在图里，路径推理就能做到。

实际开发中，大多数项目用不到完整的图数据库（Neo4j），更常见的方案是用结构化 JSON + 语义向量的混合存储：

```
// 混合记忆结构：结构化属性 + 语义描述
interface MemoryNode {
  id: string;
  type: "person" | "project" | "preference" | "event";
  attributes: Record<string, unknown>;
  relations: Array<{
    type: string; // "likes", "works_on", "knows_about"
    targetId: string;
  }>;
  embedding?: number[]; // 可选：语义向量，用于模糊检索
}

class StructuredMemoryStore {
  private nodes = new Map<string, MemoryNode>();

  // 存储或更新节点
  upsert(node: MemoryNode) {
    const existing = this.nodes.get(node.id);
    if (existing) {
      // 合并属性，不覆盖，防止丢失旧信息
      this.nodes.set(node.id, {
        ...existing,
        attributes: { ...existing.attributes, ...node.attributes },
        relations: [...new Set([...existing.relations, ...node.relations])],
      });
    } else {
      this.nodes.set(node.id, node);
    }
  }

  // 图遍历：找到所有2跳以内的相关节点
  getRelated(nodeId: string, depth = 2): MemoryNode[] {
    const visited = new Set<string>();
    const result: MemoryNode[] = [];

    const traverse = (id: string, currentDepth: number) => {
      if (currentDepth === 0 || visited.has(id)) return;
      visited.add(id);

      const node = this.nodes.get(id);
    }
  }
}
```

```
    if (!node) return;

    result.push(node);
    node.relations.forEach(({ targetId }) => {
      traverse(targetId, currentDepth - 1);
    });
  };

  traverse(nodeId, depth);
  return result;
}

// 序列化为 LLM 可读的 context
toContextString(nodeId: string): string {
  const related = this.getRelated(nodeId);
  return related
    .map((n) => `[${n.type}] ${JSON.stringify(n.attributes)}`)
    .join("\n");
}

// 使用示例
const memStore = newStructuredMemoryStore();

// 存入用户节点
memStore.upsert({
  id: "user_james",
  type: "person",
  attributes: { name: "James", role: "frontend_developer" },
  relations: [
    { type: "works_on", targetId: "project_ai_assistant" },
    { type: "prefers", targetId: "tech_react" },
  ],
});

memStore.upsert({
  id: "tech_react",
  type: "preference",
  attributes: { name: "React", category: "frontend_framework" },
  relations: [
    { type: "has_ecosystem", targetId: "tech_vercel_ai_sdk" },
  ],
});

// Agent 回答时，注入用户的知识图谱上下文
const context = memStore.toContextString("user_james");
// 输出: "[person] {\"name\":\"James\",\"role\":\"frontend_developer\"}
//       [preference] {\"name\":\"React\",\"category\":\"frontend_framework\"}
//       ..."
```


核心：知识图谱的价值在于「关系推理」，混合结构化 JSON + 向量是大多数项目的性价比最高方案。

三层记忆的协作：完整的记忆感知 Agent

把三层记忆整合起来，才是一个真正有"记忆"的 Agent：

```
import { ChatOpenAI } from "@langchain/openai";
import { SystemMessage, HumanMessage } from "@langchain/core/messages";

class MemoryAwareAgent {
  private llm = new ChatOpenAI({ model: "gpt-4o" });
  private shortTermHistory: Message[] = [];
  private longTermStore: LongTermMemoryStore;
  private knowledgeGraph: StructuredMemoryStore;

  constructor(private userId: string) {
    this.longTermStore = new LongTermMemoryStore();
    this.knowledgeGraph = new StructuredMemoryStore();
  }

  async chat(userInput: string): Promise<string> {
    // Step 1: 检索长期记忆（并行执行，不阻塞）
    const [longTermMemories, graphContext] = await Promise.all([
```

```

        this.longTermStore.recall(userInput, this.userId),
        Promise.resolve(this.knowledgeGraph.toContextString(this.userId)),
    ]);

    // Step 2: 构建系统 prompt, 注入记忆层
    const systemPrompt = this.buildSystemPrompt(longTermMemories, graphContext);

    // Step 3: 管理短期记忆 (滑动窗口)
    const managedHistory = awaitmanageHistory(this.shortTermHistory);

    // Step 4: 调用模型
    const messages = [
        newSystemMessage(systemPrompt),
        ...managedHistory,
        newHumanMessage(userInput),
    ];

    const response = awaitthis.llm.invoke(messages);

    // Step 5: 更新短期记忆
    this.shortTermHistory.push(
        { role: "user", content: userInput },
        { role: "assistant", content: response.contentasString }
    );

    // Step 6: 异步决策是否存入长期记忆 (不影响响应速度)
    this.asyncSaveMemory(userInput, response.contentasString);

    return response.contentasString;
}

privatebuildSystemPrompt(memories: string[], graphCtx: string): string {
    const parts = ["你是一个有记忆的 AI 助手。"];

    if (graphCtx) {
        parts.push(`\n关于用户你知道: \n${graphCtx}`);
    }

    if (memories.length > 0) {
        parts.push(`\n相关的历史记忆: \n${memories.join("\n")}`);
    }

    return parts.join("\n");
}

// 异步存储: 不阻塞当前响应
privateasyncasyncSaveMemory(input: string, output: string) {
    // 让模型判断这轮对话是否值得记忆
    const shouldSave = awaitthis.llm.invoke([
        newSystemMessage(

```

```
        "判断以下对话是否包含值得长期记忆的信息（用户偏好/重要决策/事实），只回答 yes 或 no"
    ),
    newHumanMessage(`用户: ${input}\n助手: ${output}`),
  ]);

  if ((shouldSave.contentAsString).toLowerCase().includes("yes")) {
    await this.longTermStore.save(
      `用户说: ${input}, 助手回答: ${output}`,
      { type: "conversation", importance: "high", userId: this.userId }
    );
  }
}
```

核心：三层记忆各司其职，检索并行化，存储异步化，才能做到有记忆而不慢。

常见坑

坑1：把所有对话都存进向量库

```
// ❌ 每轮都存，存了一堆没用的
async function onMessage(msg: Message) {
```

```
await vectorStore.addDocuments([new Document({ pageContent: msg.content })]);
}
```

向量库里全是「好的」「明白了」「那接下来呢」这类废话，检索出来全是噪音。

```
// ✅ 让模型判断是否值得存
const worthSaving = await llm.invoke([
  new SystemMessage("这句话有没有值得记忆的关键信息? yes/no"),
  new HumanMessage(msg.content),
]);
if (worthSaving.content === "yes") {
  await vectorStore.addDocuments([...]);
}
```

坑2：检索不区分用户

```
// ❌ 全局检索，用户A的记忆跑到用户B那里
const results = await vectorStore.similaritySearch(query, 5);
```

生产环境里多用户共用一个向量库，不加过滤条件会导致记忆错位。

```
// ✅ 检索时带 userId 过滤
const results = await vectorStore.similaritySearch(query, 5, {
  filter: { userId: currentUserId },
});
```

坑3：短期记忆直接截断，丢失关键上下文

```
// ❌ 超长就直接砍掉前面的
if (messages.length > 20) {
  messages = messages.slice(-20); // 可能把任务背景砍掉了
}
```

用摘要压缩，而不是硬截断：

```
// ✅ 超长时先摘要，再拼接最近内容
const summary = await summarizeOlderMessages(messages.slice(0, -20));
messages = [new SystemMessage(`历史摘要: ${summary}`), ...messages.slice(-20)];
```

坑4：记忆注入太多，反而稀释了 Prompt

```
// ❌ 检索 Top 20 条，全部塞进 prompt
const memories = await vectorStore.similaritySearch(query, 20);
const systemPrompt = `你知道: ${memories.join("\n")}`;
// 20条记忆 + 用户问题 + 对话历史 = Context 直接爆炸
```

```
// ✅ 控制注入数量，只用相关性 > 0.8 的，最多 5 条
const results = await vectorStore.similaritySearchWithScore(query, 5);
const relevant = results
  .filter(([_ , score]) => score > 0.8)
  .map(([doc]) => doc.pageContent);
```

坑5：忘记给记忆加过期机制

记忆不应该永久有效。用户一年前说「我不喜欢 Vue」，不代表现在还这样。

```
// ✅ 存储时加 TTL，检索时跳过过期记忆
await vectorStore.addDocuments([
  new Document({
    pageContent: content,
    metadata: {
      timestamp: Date.now(),
      ttlDays: 90, // 90天后过期
    },
  }),
]);

// 检索时过滤：只取 90 天内的记忆
const cutoff = Date.now() - 90 * 24 * 60 * 60 * 1000;
const results = await vectorStore.similaritySearch(query, 5, {
  filter: { timestamp: { $gt: cutoff } },
});
```

选型参考

需求	推荐方案	备注
单用户简单 chatbot	MemoryVectorStore (LangChain 内存版)	开发测试用，不持久化

需求	推荐方案	备注
多用户生产环境	Chroma / Pinecone / Weaviate	Chroma 本地部署友好，Pinecone 托管省心
需要关系推理	Neo4j + 向量混合	复杂度高，一般 Agent 不需要
企业内网知识库	pgvector (PostgreSQL 插件)	已有 PG 的团队最低迁移成本

发布前自查清单

- 短期记忆有滑动窗口 + 摘要压缩，不是无脑追加
- 向量存储检索加了用户隔离过滤
- 存储前有重要性判断，不存废话
- 相关性阈值 > 0.7，注入数量 ≤ 5 条
- 记忆有 TTL，不永久有效
- 长期记忆存储是异步的，不阻塞当前响应

总结

这篇我们从底层拆解了 Agent 记忆系统的三层设计：

- **短期记忆**：Context Window 不是越满越好，滑动窗口 + 摘要压缩是正确姿势
- **长期记忆**：向量数据库实现语义检索，关键是选择性存储 + 时间衰减
- **知识图谱**：解决关系推理问题，混合 JSON + 向量是大多数项目的性价比最优解
- **三层协作**：检索并行化、存储异步化，做到有记忆而不慢
- **常见坑**：存太多、不隔离用户、硬截断、注入太多、忘记过期——五个坑踩过才知道

理解记忆系统的核心是：**记忆本质上是 context 管理的延伸，每一层记忆都是在回答"什么信息值得在什么时候出现在 prompt 里"这个问题。**

下一篇我们进入 LangGraph，聊聊当 Agent 任务复杂到「一条链跑不完」的时候，为什么要从链升级到图。

关注我，James 的成长日记，持续分享干货，帮你在 AI 时代少走弯路。

喜欢作者

AI Agent 修炼路 · 目录

< 上一篇

下一篇 >

ReAct 模式拆解：Agent 如何做到“边想边做”；

LangGraph 是什么：复杂 Agent 为何要从链升级到图

作者提示: 内容由AI生成